University of Glasgow
James Watt School of Engineering

Robotics Team Design Project M (ENG5325)

Cyberphysical RoboCup Soccer Teams

Final technical report

Team:	Team 14
Manager:	Dr Euan McGookin
Supervisor:	Ziang Cheng
Submission date:	April 2026

Nathan Sidi Bakari	Team lead / Communication and management officer
Linhan Ma	2D simulation sub-team lead
Wei Cui	3D simulation sub-team lead
Zhenkun Pang	2D simulation sub-team member
Liyue Tian	3D simulation sub-team member

Author contributions

This report was prepared collaboratively by all members of Team 14. The principal authorship of each main section is summarised in table 1. Final editing, formatting, cross-referencing, and integration were performed jointly to ensure a consistent technical narrative.

Table 1: Indicative authorship of the main report sections.

Section	Primary contributing author(s)
Abstract and Introduction	Nathan Sidi Bakari
Methodology: 2D simulation and benchmarking	Linhan Ma, Zhenkun Pang
Methodology: 3D simulation and benchmarking	Wei Cui, Liyue Tian
Results and analysis	All team members
Team performance analysis	Nathan Sidi Bakari
Conclusions and further work	All team members
Appendix	All team members

Acknowledgements

Team 14 gratefully acknowledges the guidance of Dr Euan McGookin and Ziang Cheng throughout the project. Their feedback during supervision and review meetings helped the team identify that the original strategy-level comparison needed to be extended into a parameter-level OFAT analysis. The team also acknowledges the FC Portugal, SimSpark, RoboViz, RoboCup 2D soccer simulation, and open-source base-team communities, whose software made the simulation and benchmarking work possible.

Abstract

This report presents the design, implementation, and evaluation of simulation-based RoboCup humanoid soccer teams developed for Robotics Team Design Project M (ENG5325). The project focused on Stage 1 of the assignment: a behavioural benchmarking framework for autonomous teams operating in 2D and 3D simulation environments derived from established RoboCup software stacks. Strategy-level benchmarks first compared bundled team behaviours. Parameter-level experiments were then added to isolate the behavioural mechanisms associated with performance changes. The evidence base combines automated benchmark execution, logged simulation outputs, summarised result tables, and role-based interpretation of the controller logic for strikers, defenders, and goalkeepers. The results indicate that aggressive strategy settings performed best among the tested variants, while parameter-level analysis made those differences easier to interpret. The report also evaluates project planning, time and cost management, and team coordination. Although the project did not progress to hardware implementation, the completed simulation and analysis pipeline provides a basis for future transfer to physical NAO6 platforms.

Contents

1 Introduction **1**

1.1 Project overview . . . 1
1.2 Project requirements . . . 1
1.3 Key requirements and scope boundary . . . 1
1.4 Engineering context . . . 2
1.5 Aim and objectives . . . 2
1.6 Report structure . . . 3

2 Methodology **3**
2.1 Project workflow and design logic . . . 3
2.2 Stage boundary and requirement traceability . . . 4
2.3 Simulation environments and technical baseline . . . 5
2.3.1 Common agent-server structure . . . 5
2.3.2 2D environment . . . 6
2.3.3 3D environment . . . 6
2.3.4 Rationale for platform choice . . . 6
2.4 Role-based behaviour framework . . . 7
2.4.1 Finite-state interpretation of roles . . . 8
2.5 Strategy-level benchmarking design . . . 9
2.6 Parametric benchmarking and OFAT design . . . 9
2.6.1 Benchmark modes . . . 10
2.7 Metrics and evidence framework . . . 11
2.8 Automation, data extraction, and repeatability . . . 11
2.9 Data consolidation and post-processing . . . 12
2.10 Validation and limitations of the methodology . . . 12

3 Results and analysis **12**
3.1 Strategy-level team performance . . . 12
3.2 Negative control and baseline variance . . . 13
3.3 Environment-dependent behaviour separation . . . 14
3.4 Striker parameter analysis . . . 14
3.5 Defender parameter analysis . . . 15
3.6 Goalkeeper parameter analysis . . . 16
3.7 Cross-role parametric synthesis . . . 17
3.8 Interim technical interpretation . . . 18

4 Team performance analysis **19**

4.1 Team structure and work allocation . . . 19

4.2 Planning, review, and iteration . . . 21

4.3 Estimated versus actual time and cost . . . 22

4.4 Management evaluation against assessment criteria . . . 23

4.5 Evaluation of team performance . . . 25

4.6 Lessons learned . . . 25

5 Conclusions and further work **25**

5.1 Conclusions . . . 25

5.2 Overall assessment of project outcomes . . . 26

5.3 Further work . . . 26

A Benchmark command sheet **29**

A.1 2D commands . . . 29

A.2 3D commands . . . 30

B Benchmark output structure **31**

C Benchmark summary tables **32**

D Workflow notes and limitations **33**

List of Figures

1 Overall workflow from simulator selection to strategy benchmarking and parameter-level behavioural analysis. . . . 4

2 Conceptual role-based controller framework linking striker, defender, and goalkeeper behaviours. . . . 8

3 Strategy-level comparison across 2D and 3D environments. . . . 13

4 Striker-focused parameter analysis. . . . 15

5 Defender-focused parameter analysis. . . . 16

6 Goalkeeper and team-defence parameter analysis. . . . 17

7 Planned high-level WBS Gantt chart for the Stage 1 project schedule, grouped by major work package. . . . 21

8 Estimated and actual project effort and labour-equivalent cost by workstream. . . . 22

9 Total estimated versus actual effort and labour-equivalent cost. . . . 23

List of Tables

1 Indicative authorship of the main report sections. i
2 Project brief requirements and how they are treated in this report. 2
3 How the Stage 1 brief is operationalised in the report. 5
4 Role-level behavioural interpretation used in the methodology. 8
5 Strategy-level benchmarking plan. 9
6 Parameter-level design used for local sensitivity analysis. 10
7 Benchmark modes used in the parametric workflow. 11
8 Summary of the main performance metrics used in the report. 11
9 Strategy-level performance against BASIC across repeated benchmark runs. . . . 13
10 Separation between strategy variants by environment. 14
11 Striker parameter results computed from repeated benchmark runs. 15
12 Defender parameter results computed from repeated benchmark runs. 16
13 Goalkeeper and team-defence results computed from repeated benchmark runs. . 17
14 Cross-role synthesis of the main parameter-level findings. 18
15 Main project workstreams and management observations. 20
16 Estimated and actual effort by workstream from the project records. 22
17 Evaluation of project performance against the management criteria implied by the brief and assessment matrix. 24
18 Benchmark output locations. 31
19 Common files generated by parametric benchmark runs. 31
20 Strategy benchmark summary against BASIC. 32
21 Striker parameter summaries. 32
22 Defender parameter summaries. 32
23 Goalkeeper and team-defence parameter summaries. 33

1 Introduction

1.1 Project overview

The aim of this project was to design and evaluate cyberphysical RoboCup soccer teams within a simulation-based engineering framework. The brief required the development of virtual humanoid soccer teams based on the RoboCup KidSize format and intended eventual transfer of behavioural algorithms to NAO6 hardware platforms. In practice, the present report focuses on Stage 1 of the project, namely simulation development, behavioural design, benchmarking, and performance analysis [1].

RoboCup is a suitable project context because robot soccer was originally proposed as a benchmark for real-time, uncertain, cooperative, and adversarial AI and robotics research [2, 3]. The modern RoboCup soccer leagues also deliberately span multiple levels of physical realism, from abstract simulation environments to embodied humanoid platforms [4]. This makes simulation-based team development a relevant first stage for studying behaviour before attempting transfer to physical robots.

1.2 Project requirements

The project specification required the team to consider the design of two competing teams of autonomous humanoid robots, the simulated playing environment, and the behavioural logic required for soccer play. The task extended beyond visual simulation alone and included modelling, control, guidance, algorithm development, testing, project planning, and cost estimation.

1.3 Key requirements and scope boundary

The module specification makes the project more specific than a generic robot-soccer exercise. It defines a two-stage engineering task: Stage 1 is the creation of a simulation of two competing teams and their playing environment, together with the design and evaluation of the associated behavioural algorithms; Stage 2 is the transfer of those algorithms onto humanoid hardware [1]. The brief also constrains the problem to autonomous humanoid teams operating within a RoboCup-style football environment, with role-sensitive behaviour for strikers, defenders, and goalkeepers, and with explicit project-management deliverables such as timing, cost estimation, and continual time-allocation reporting.

These requirements are consistent with the RoboCup Humanoid League material cited for this project, where KidSize competition emphasises autonomous soccer skills, behaviour design, localisation, walking, and team play under a FIFA size-1 ball and a four-robot team format [5, 6]. Within the wider RoboCup ecosystem, the NAO platform also provides an embodied reference point through the Standard Platform League, where software capability, perception, locomotion, and team coordination are evaluated under a common hardware baseline [7]. The resulting scope boundary is summarised in table 2, which shows how each requirement from the brief is treated in the present report rather than left as an implicit assumption.

Table 2: Project brief requirements and how they are treated in this report.

Requirement from brief	Interpretation in the present report
Two-stage assignment	The report concentrates on Stage 1 simulation development, benchmarking, and analysis; Stage 2 hardware deployment is treated as future transfer work rather than as a completed deliverable.
Autonomous humanoid teams	The methodology focuses on autonomous agent decision loops, role logic, and environment interaction without external operator intervention.
NAO6-inspired KidSize setting	The work uses 2D and 3D RoboCup-derived simulators as complementary abstractions for rapid comparison and more embodied validation.
Role-sensitive football behaviour	Striker, defender, and goalkeeper roles are used as the main interpretive framework for controller analysis and parameter-level results.
Project-management deliverables	Time allocation, cost estimation, planning, and team performance are treated as first-class report content rather than as appendices to the technical work.

1.4 Engineering context

This project sits at the intersection of robotics, multi-agent systems, simulation-based design, and performance evaluation. The engineering challenge was not just to make two teams move on a pitch, but to test whether behaviour configurations produced measurable differences in team and role performance across both 2D and 3D environments. This framing follows the wider view that simulation is valuable when it provides a controlled and repeatable testbed for robotics design and validation [8, 9].

1.5 Aim and objectives

The overall aim of this work was to construct and evaluate a simulation-based behavioural benchmarking pipeline for RoboCup soccer teams. The principal objectives were:

1. to establish workable 2D and 3D simulation environments based on validated existing codebases;
2. to define a set of strategy variants representing different team behaviours;
3. to benchmark those variants using repeated automated matches;
4. to isolate selected behavioural parameters for role-informed sensitivity analysis;
5. to compare performance using team-level outcome metrics and secondary role-related indicators; and
6. to assess team performance, project planning, and time/cost management.

Within those objectives, the main contribution of the present report is not a claim of full humanoid-hardware deployment. It is the Stage 1 evidence chain: workable 2D and 3D simulation environments, a repeatable behavioural benchmarking workflow, role-informed parameter analysis, and a project-management evaluation that explains what was delivered and why the final scope took this form.

1.6 Report structure

This report follows the structure required by the module guidelines and sound engineering practice. Section 2 describes the implementation and evaluation workflow, section 3 presents the benchmark evidence, section 4 evaluates team performance and project management, and section 5 summarises the principal technical findings and areas for further work.

2 Methodology

2.1 Project workflow and design logic

The project was organised as a staged simulation-and-analysis workflow, not as a single demonstration. First, the team established working 2D and 3D RoboCup environments from existing community stacks. Next, controlled behaviour variants were introduced and complete matches were run repeatedly. Strategy-level benchmarking then compared complete behaviour bundles against a fixed baseline. The final stage isolated individual behavioural parameters around that baseline so that the results could be interpreted by mechanism rather than by strategy label alone.

This sequence mattered because whole-strategy comparisons can identify which bundle performs better, but they do not explain which decision rule caused the difference. After the design review stage, the methodology was refined from “which strategy wins?” towards “which parameter changes behaviour, and in which role does that effect appear?” The workflow linked implementation choices to measured outcomes while staying within the time and compute limits of the project. Figure 1 shows this sequence from simulator setup and controller configuration to strategy screening, parameter isolation, and role-level interpretation. This is consistent with the sensitivity-analysis literature, where local parameter studies can support mechanism discovery but must be interpreted more cautiously than global designs [10, 11].

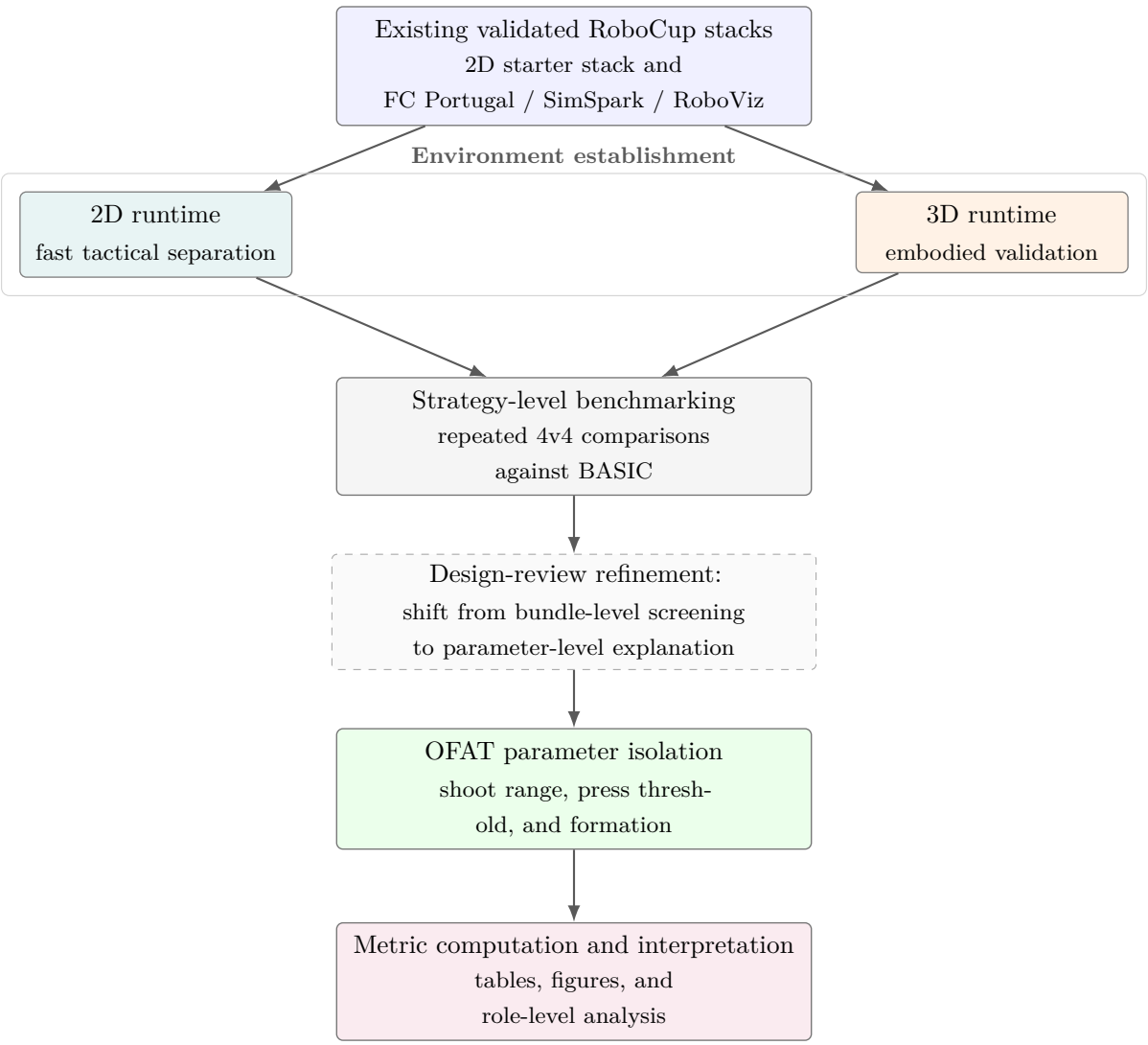


Figure 1: Overall workflow from simulator selection to strategy benchmarking and parameter-level behavioural analysis.

2.2 Stage boundary and requirement traceability

The module brief and the design-review gate meant that Stage 1 had to demonstrate more than a visually plausible match. In practical terms, the project needed to show that usable simulation environments had been established, that autonomous team behaviours could be implemented and exercised within those environments, and that the resulting outputs could support technical interpretation and comparison [1, 12]. The workflow therefore followed a V-model-like progression from requirements and platform selection through modelling, controller development, benchmarking, review, and reporting, with the design review acting as the main stage gate before any notional progression to hardware.

For this reason, Stage 1 completion is treated in this report as a combination of environment setup, controller implementation, repeated benchmark execution, and evidence-based interpretation. A simulation-only deliverable without repeatable behavioural evidence would not have satisfied the intent of the assignment as stated in the brief. Table 3 makes this interpretation auditable by

linking each Stage 1 requirement area to the report evidence used later.

Table 3: How the Stage 1 brief is operationalised in the report.

Stage 1 requirement area	Response in this report	Main evidence location
Operational simulation environments	Establish 2D and 3D runtime paths from validated RoboCup stacks rather than constructing a simulator from first principles.	Methodology, simulator subsections and appendix command sheet.
Autonomous team behaviour	Implement role-sensitive control logic and strategy/parameter variants within the agent decision layer.	Role framework, strategy design, and parameter-design subsections.
Behaviour testing and analysis	Use repeated strategy benchmarking and OFAT parameter sweeps to generate comparative evidence.	Results section and benchmark summary tables.
Design-review readiness	Treat Stage 1 as a sign-off boundary for simulation and behavioural analysis, not as implicit completion of hardware transfer.	Workflow discussion, limitations, and conclusions/further work.
Project-management accountability	Track work allocation, time, and labour-equivalent cost and evaluate the effectiveness of the team process.	Team performance analysis and TAR-derived summary tables.

2.3 Simulation environments and technical baseline

2.3.1 Common agent-server structure

Both simulation environments used the same high-level control principle. The server advanced the match clock and applied the physical and referee rules, while each robot client independently received state information, updated its internal world model, selected an action, and sent a command back to the server. This separation meant that the project work could focus on decision logic and experimental orchestration without modifying the simulator physics or referee implementation. The RoboCup soccer server has been described in the literature as shared multi-agent research infrastructure, which supports this separation between environment implementation and agent behaviour [13].

At each simulation cycle, the agent loop can be summarised as perception, world-model update, role decision, command generation, and server update. This structure is central to the methodology because it isolates behavioural changes from environment mechanics. When a strategy or

parameter was changed, the intention was to alter the agent decision layer, not the match rules.

2.3.2 2D environment

The 2D environment was based on the RoboCup 2D soccer server and an inherited starter-stack/Helios-style agent framework. The official soccer-simulator manual and the infrastructure literature both emphasise the server/client split, including objects, protocols, sensor models, action models, and referee logic, which aligns with the way this project used the 2D stack as a controlled benchmarking environment [13, 14]. The server was responsible for the pitch, ball, players, timing, collisions, play modes, and score. The agent code handled parsing of sensory messages, selection of high-level behaviours, and serialisation of commands back to the server. The use of a Helios-style base is consistent with the 2D simulation community's practice of building on open source base teams and libraries rather than recreating the entire infrastructure [15].

Within the 2D stack, the decision code used for this project was concentrated in the sample player and basic behaviour modules. These modules map strategy settings onto tactical decisions such as when to shoot, when to press, when to support shape, and when to clear or pass. Benchmark scripts launched the server and teams, collected match outcomes, and wrote structured outputs for later analysis. The 2D environment was used as the faster, more abstract environment for repeated behavioural comparison.

2.3.3 3D environment

The 3D environment was based on the FC Portugal codebase, SimSpark, and RoboViz. SimSpark and `rcssserver3d` provided the physics and match control. The FC Portugal agent framework provided the robot client structure, including world modelling, communication, path planning, behaviour selection, and command generation. RoboViz was used for visualisation and for the log stream from which match-level statistics could be parsed; this is consistent with its published role as a real-time visualisation and debugging tool for complex multi-agent soccer scenes [16].

The 3D environment added embodied robot dynamics, actuation limits, falling and recovery behaviour, and slower match execution. These properties made it closer to the intended humanoid setting than 2D, but also made outcomes noisier and more draw-dominated. Prior comparisons of humanoid RoboCup platforms emphasise that 3D simulation is closer to NAO-style embodiment than 2D abstraction, but still remains a simulation environment with transfer limitations [17, 18]. The implemented workflow should be understood as RoboViz-supported simulation-based validation, not as a fully headless execution pipeline.

2.3.4 Rationale for platform choice

Established codebases were adopted so that effort could be spent on behaviour design, benchmark orchestration, parameter isolation, and analysis rather than low-level simulator infrastructure. This was an engineering trade-off: building a simulator from first principles would have consumed effort without directly answering the behavioural evaluation question. The literature supports this approach: RoboCup infrastructure and open base teams are intended to provide reusable

platforms for comparative multi-agent research [13, 15], and simulation leagues are valued for repeatable experimentation [8]. In this report the term simulation-based validation is preferred over digital-twin language, because no physical NAO6 hardware loop was completed during the project.

No core physics engine or referee logic was modified. The project changes were confined to the agent decision layer, strategy and parameter configuration, benchmark launch scripts, and result extraction. This kept the experiments grounded in standard RoboCup simulation environments while still allowing controlled behavioural comparison.

2.4 Role-based behaviour framework

The team behaviour was analysed through three principal functional roles: striker, defender, and goalkeeper. The purpose of this role-based framework was not to claim that each role was isolated perfectly during a match, but to make the decision logic interpretable enough to connect parameter changes to football behaviour. Role-sensitive coordination is a common theme in robotic soccer, where dynamic assignment and positioning are used to connect local robot decisions to team shape [19].

In the 2D system, role allocation was primarily slot-based. The goalkeeper occupied the goalkeeper slot, one field player acted as the main defender, and the remaining field players acted as attacking/support players. Local dynamic overrides still occurred: for example, a non-goalkeeper could become the immediate ball-winner if its intercept condition was satisfied, and set plays could temporarily reassign the kicker.

In the 3D system, role activation was more dynamic. The framework estimated which teammate was closest or best placed relative to the predicted ball state. That active player was routed through the attacking/striker logic, while non-active players adopted goalkeeper or support/defender behaviour depending on their uniform number and context. This made 3D role behaviour more dependent on local game state than on a fixed slot alone. Figure 2 shows how these role routes connect to the shared decision loop, and table 4 defines the 2D and 3D interpretation used when the results are later grouped by striker, defender, and goalkeeper.

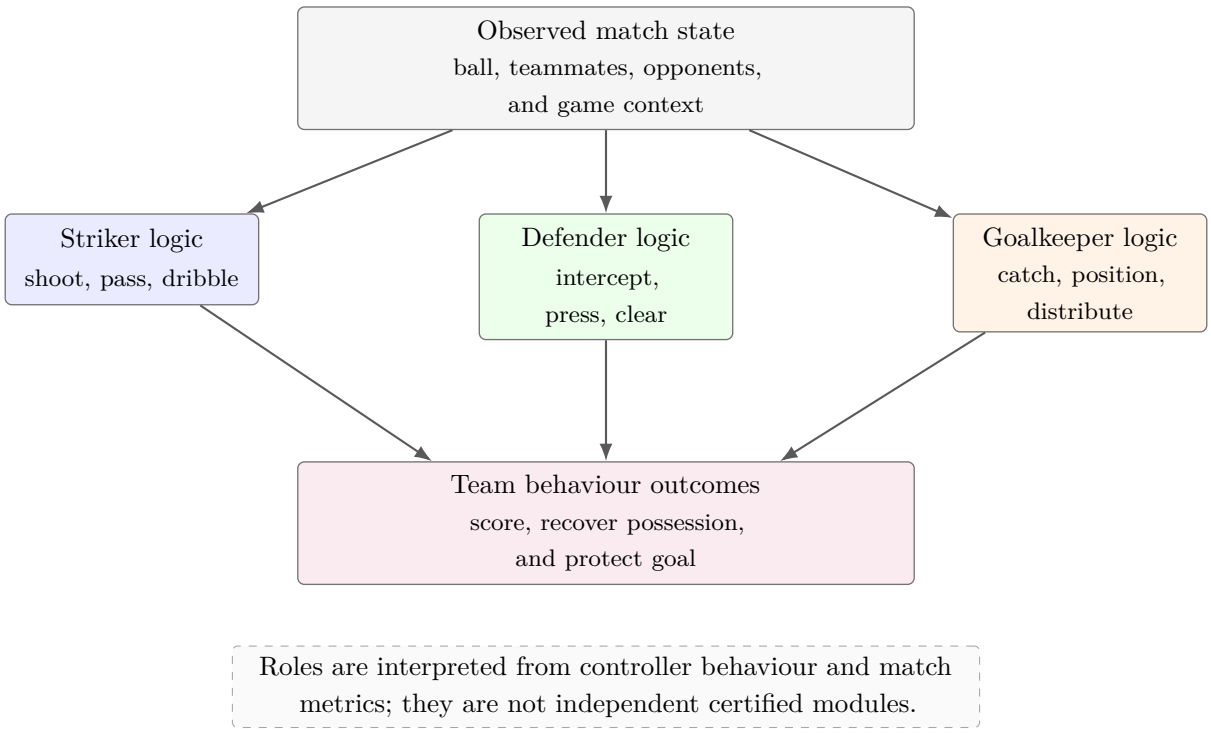


Figure 2: Conceptual role-based controller framework linking striker, defender, and goalkeeper behaviours.

Table 4: Role-level behavioural interpretation used in the methodology.

Role	2D interpretation	3D interpretation
Striker	Field player currently attacking, shooting, chasing, or supporting the ball.	Active player selected around the predicted ball state and routed through attacking behaviour.
Defender	Field player anchored to a defensive/support slot, with dynamic interception when appropriate.	Non-active, non-goalkeeper player maintaining support shape and defensive positioning.
Goalkeeper	Goalkeeper slot responsible for catch, distribution, and goal-line positioning.	Goalkeeper hold policy by default, with fallback to active-player behaviour if selected by the game state.

2.4.1 Finite-state interpretation of roles

For analysis, the role logic was interpreted as finite-state behaviour. In 2D, the striker-like behaviour moved between preprocessing, attacking with the ball, chasing/supporting, and set-play kicking. The main attacking hierarchy was to shoot if inside the configured shooting range, otherwise pass, dribble, hold, or clear depending on pressure and space. The defender used similar field-player primitives but with a more defensive anchor and a greater emphasis on interception, clearing, and shape. The goalkeeper moved between catching, distributing, and goal-line positioning.

In 3D, the striker controller was more explicit: search for the ball, approach, align, kick, and recover. Non-active field players mainly used support-shape behaviour, while the goalkeeper normally held a goal position and faced the ball. Strategy and parameter settings changed thresholds and target choices inside these policies; they did not replace the entire FSM topology.

2.5 Strategy-level benchmarking design

The first evaluation layer compared bundled behaviour configurations at the team level. The fixed reference was BASIC. The tested variants were NOISE, DEFLOCK, HIPRESS, DIRECT, and AGGRO, each benchmarked against BASIC in both 2D and 3D. Strategy benchmarking used full 4v4 matches, which matched the project convention of three field players plus one goalkeeper per side.

The strategy names were compact labels for grouped parameter choices. Each label represented a football intention: baseline behaviour, a duplicate baseline for variance checking, more conservative defence, higher pressing, more direct forward play, or more aggressive attacking. Table 5 states the strategy set, repetition count, and environment coverage used for this screening stage. The limitation is that a strategy bundle changes multiple behavioural tendencies at once, so a win-rate difference does not by itself identify the decision parameter responsible.

Table 5: Strategy-level benchmarking plan.

Strategy	Behavioural intent	Sides	Repetitions	Environments
BASIC	Baseline reference controller	2	5	2D, 3D
NOISE	Duplicate of baseline for variance checking	2	5	2D, 3D
DEFLOCK	More conservative, defence-oriented behaviour	2	5	2D, 3D
HIPRESS	Earlier pressing and stronger ball contesting	2	5	2D, 3D
DIRECT	More direct transitions and forward play	2	5	2D, 3D
AGGRO	More aggressive attack and stronger scoring intent	2	5	2D, 3D

This stage was diagnostic. Strategy benchmarking tested whether the pipeline could detect behavioural separation between complete team configurations. It also motivated the parameter-isolation stage.

2.6 Parametric benchmarking and OFAT design

A second evaluation layer varied one factor at a time around the BASIC baseline. This one-factor-at-a-time (OFAT) design was chosen because the project needed interpretable behavioural evidence more than an exhaustive search of the whole strategy space. Holding the baseline fixed and changing only one parameter made it easier to connect an observed change in goals,

recoveries, or shot exposure to a specific decision threshold. This use of OFAT is defensible as a local, mechanism-oriented sensitivity method, especially in agent-based settings, but it does not provide the interaction coverage of global sensitivity analysis [10, 11].

The selected parameters were shooting range, pressing threshold, and formation identifier. Shooting range changed when an attacker was willing to shoot. Pressing threshold changed how readily players contested the ball. Formation changed the spatial support structure of the team. The first two parameters were threshold changes; formation was less isolated because it changed spacing and interactions between players, but it was still needed to explain defender and goalkeeper outcomes. In the benchmark scripts these factors were implemented using explicit internal values rather than only verbal labels. Shoot range low / baseline / high corresponds to `20.0 / 25.0 / 30.0`; press threshold low / baseline / high corresponds to `2 / 3 / 4`; and formation defensive / baseline / offensive corresponds to `DEF_121 / BASELINE / OFF_112`. Table 6 records this mapping so the result labels remain traceable to configured benchmark values.

Table 6: Parameter-level design used for local sensitivity analysis.

Parameter	Qualitative labels used in results	Configured values used in benchmarks	Primary role(s) affected
Shooting range	Low / baseline / high	`20.0 / 25.0 / 30.0`	Striker
Pressing threshold	Low / baseline / high	`2 / 3 / 4`	Striker, defender
Formation identifier	Defensive / baseline / offensive	`DEF_121 / BASELINE / OFF_112`	Defender, goalkeeper, team structure

2.6.1 Benchmark modes

Parametric benchmarking used smaller match configurations where they helped isolate behaviour. The 1v1 mode was treated as a no-goalkeeper microbenchmark: it supports fast field-player checks, but it is not representative of normal goal-protection behaviour. The 2v2 mode was the preferred duel-style project benchmark because each side includes one field player and one goalkeeper. The 4v4 mode was used for team-level checks and for comparison with strategy benchmarking. Table 7 explains why some role interpretations are stronger in 2v2 or 4v4 than in 1v1.

Table 7: Benchmark modes used in the parametric workflow.

Mode	Interpretation	Main use
1v1	One field player per side, no goalkeepers.	Lightweight microbenchmark for isolated player behaviour.
2v2	One field player plus one goalkeeper per side.	Preferred duel-style benchmark because goalkeeping is present.
4v4	Three field players plus one goalkeeper per side.	Team-level benchmark and strategy comparison setting.

Formation experiments are most informative in 4v4 because formation is a team-shape parameter. They can still be run in smaller modes for pipeline validation, but the interpretation is limited when there are too few field players to express a full team shape.

2.7 Metrics and evidence framework

The evaluation used team-level outcome metrics together with role-related process indicators. Team-level metrics included win rate, draw rate, goals scored, goals conceded, and goal difference. These were the most comparable cross-environment metrics because they are tied to match outcomes.

The role-related indicators computed from the benchmark outputs included scoring rate, time to score, possession, recoveries, interceptions, clearances, defensive stability, shots conceded, and goalkeeper outcomes. These indicators were used to interpret behaviour changes rather than to claim exhaustive role certification. For example, striker analysis focused on scoring and time-to-score trends; defender analysis focused on recoveries, interceptions, clearances, and defensive stability; goalkeeper analysis was interpreted through save rate and goals conceded, while recognising that goalkeeper outcomes depend strongly on upstream defence. Table 8 defines this metric-to-role mapping and is therefore the basis for the structure of the results subsections.

Table 8: Summary of the main performance metrics used in the report.

Level / role	Metrics
Team	Win rate, draw rate, goals scored, goals conceded, goal difference
Striker	Goals per match, time to score, possession-related indicators
Defender	Recoveries, interceptions, clearances, defensive stability indicators
Goalkeeper	Save rate, shots conceded, goal-prevention indicators

2.8 Automation, data extraction, and repeatability

Benchmark scripts were used to run matches, collect per-run outputs, and reduce manual inconsistency. In 2D, the benchmark scripts launched the server and teams, controlled the number of players, applied strategy or parameter settings, and parsed the final score into structured output. In 3D, the benchmark scripts launched the SimSpark/RoboViz-backed

workflow, monitored match completion, parsed RoboViz logs, and recorded status information such as successful completion or timeout. Repeated evaluation matters in RoboCup simulation because match outcomes can be stochastic and tournament-style results can be sensitive to evaluation format [8, 20].

2.9 Data consolidation and post-processing

Benchmark runs were executed across more than one development machine, so the output CSVs and logs were collected and combined before analysis. A spreadsheet was used as a practical consolidation tool to align run metadata, compute team-level and role-level summaries, and generate the values used in the report tables and figures. The evidence base remains the simulation outputs produced by the benchmark scripts; that file only organises and post-processes those outputs.

2.10 Validation and limitations of the methodology

The adopted methodology offered repeatable comparison across multiple strategy and parameter settings, but it also had constraints. First, the project relied on inherited simulation stacks rather than a simulator developed from first principles. This was acceptable for the project aim, but it means the contribution is in behavioural configuration, benchmarking, and analysis rather than simulator physics.

Second, strategy bundles were informative for screening but insufficient for clean causal attribution. This is why the project introduced OFAT parameter isolation. Third, OFAT studies are local sensitivity checks rather than a full factorial design, so parameter interactions were not exhaustively tested. The method can say that changing shooting range, pressing threshold, or formation is associated with the observed changes under the tested baseline, but it should not be read as a complete global optimisation of all possible behaviours [10, 11].

Fourth, the 3D environment introduced greater behavioural uncertainty due to embodied dynamics, actuation limits, slower execution, and a larger fraction of draw outcomes. This is a normal limitation of simulation-based robotics work: richer simulators can improve relevance, but they also introduce modelling assumptions and do not remove the simulation-to-reality gap [9, 18]. The report presents summaries rather than every per-run CSV row; raw logs remain supporting reproducibility material.

3 Results and analysis

3.1 Strategy-level team performance

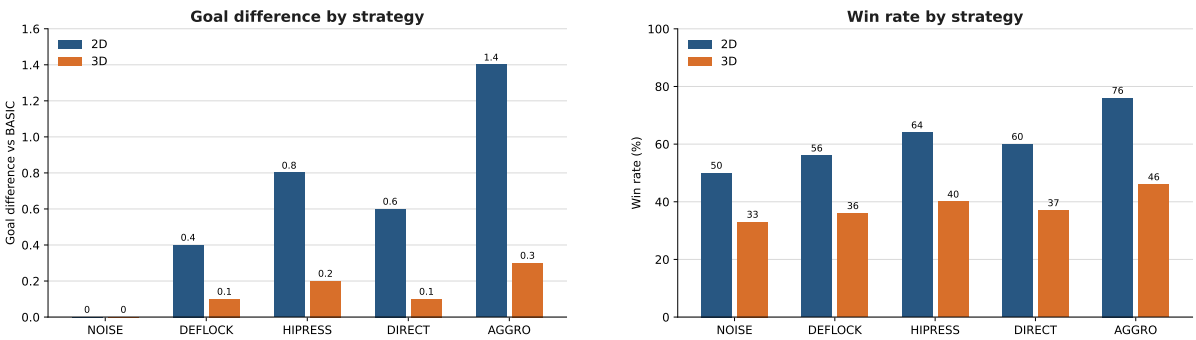
The strategy-level benchmarks provided the first comparison of team performance across repeated runs. Table 9 summarises goal difference and win rate for each tested strategy against the fixed BASIC reference. AGGRO produced the best result in both environments: a goal difference of 1.4 and win rate of 76% in 2D, and a goal difference of 0.3 and win rate of 46% in 3D.

The remaining strategies showed smaller improvements. HIPRESS was the second-best strategy

by goal difference in both environments, while DIRECT produced a moderate gain in 2D but little separation in 3D. DEFLOCK was positive but weaker than HIPRESS and AGGRO. NOISE ended at 0.0 goal difference in both environments. Because NOISE was implemented as a duplicate-baseline alias of BASIC, it served as a baseline-variance check rather than as an independent strategy result. Table 9 gives the numerical comparison, and figure 3 plots the same pattern: figure 3a shows goal-difference separation, while figure 3b shows the corresponding win-rate separation.

Table 9: Strategy-level performance against BASIC across repeated benchmark runs.

Strategy	2D goal diff.	3D goal diff.	2D win rate (%)	3D win rate (%)
NOISE	0.0	0.0	50	33
DEFLOCK	0.4	0.1	56	36
HIPRESS	0.8	0.2	64	40
DIRECT	0.6	0.1	60	37
AGGRO	1.4	0.3	76	46



(a) Goal difference. **(b)** Win rate.

Figure 3: Strategy-level comparison across 2D and 3D environments.

3.2 Negative control and baseline variance

The BASIC versus NOISE comparison served as a negative-control concept. In both the 2D and 3D implementations, NOISE mapped directly to the BASIC controller configuration, so its purpose was to test duplicate-baseline variation rather than to represent a genuinely different behaviour bundle.

The benchmark summaries place NOISE at 0.0 goal difference in both environments. That value supports baseline equivalence, but it should not be read as proof that every BASIC–NOISE run was exactly balanced. The raw benchmark rows still show match-to-match variation, especially in 2D, even though the controller settings are the same. The negative-control result shows that relabelling the baseline did not produce a systematic performance shift.

3.3 Environment-dependent behaviour separation

The repeated runs showed clearer separation between strategy variants in 2D than in 3D. In 2D, the spread in goal difference across tested strategies was 1.4, from 0.0 for NOISE to 1.4 for AGGRO. In 3D, the same spread was only 0.3. The win-rate spread showed the same pattern: 26 percentage points in 2D and 13 percentage points in 3D.

This supports the report's interpretation that 2D is the cleaner environment for rapid behavioural separation, while 3D compresses results because embodied dynamics, actuation limits, falling/recovery behaviour, and slower play reduce scoring frequency. Table 10 condenses this argument into the two separation measures used here: goal-difference range and win-rate range. This interpretation is also consistent with the distinction between abstract simulation leagues and more embodied humanoid simulation environments in the RoboCup literature [4, 17]. The separation measures do not by themselves prove every mechanism behind this compression, but they are consistent with the observed project experience that 3D matches were harder to separate by strategy.

Table 10: Separation between strategy variants by environment.

Separation measure	2D	3D
Goal-difference range	1.4	0.3
Win-rate range (pp)	26	13
Best strategy by goal difference	AGGRO	AGGRO

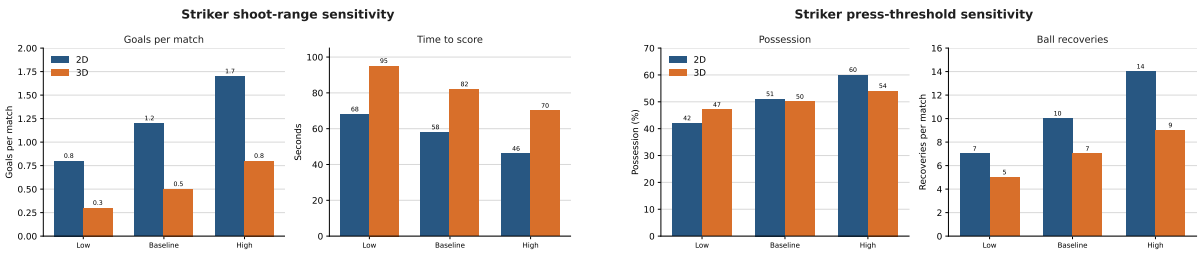
3.4 Striker parameter analysis

The striker-focused parameter results show the most direct trend. Increasing shoot range from low to high increased goals per match from 0.8 to 1.7 in 2D and from 0.3 to 0.8 in 3D. It also reduced time to score from 68 s to 46 s in 2D and from 95 s to 70 s in 3D. In the tested set, shoot range had the largest effect on attacking output.

Press threshold also affected striker-related process indicators, but the effect was weaker in 3D. In 2D, possession increased from 42% to 60% as press threshold moved from low to high, and recoveries increased from 7 to 14 per match. In 3D, possession increased from 47% to 54%, while recoveries increased from 5 to 9. This suggests that pressing affects ball recovery and control, but that the 3D environment dampens the effect. Table 11 reports the striker values, while figure 4 highlights the two main trends: figure 4a shows that higher shoot range improves scoring output, and figure 4b shows that higher pressing has a clearer effect in 2D than in 3D.

Table 11: Striker parameter results computed from repeated benchmark runs.

Parameter	Metric	Env.	Low	Baseline	High
Shoot range	Goals per match	2D	0.8	1.2	1.7
Shoot range	Goals per match	3D	0.3	0.5	0.8
Shoot range	Time to score (s)	2D	68	58	46
Shoot range	Time to score (s)	3D	95	82	70
Press threshold	Possession (%)	2D	42	51	60
Press threshold	Possession (%)	3D	47	50	54
Press threshold	Recoveries/match	2D	7	10	14
Press threshold	Recoveries/match	3D	5	7	9



(a) Shoot-range sensitivity. **(b)** Press-threshold sensitivity.

Figure 4: Striker-focused parameter analysis.

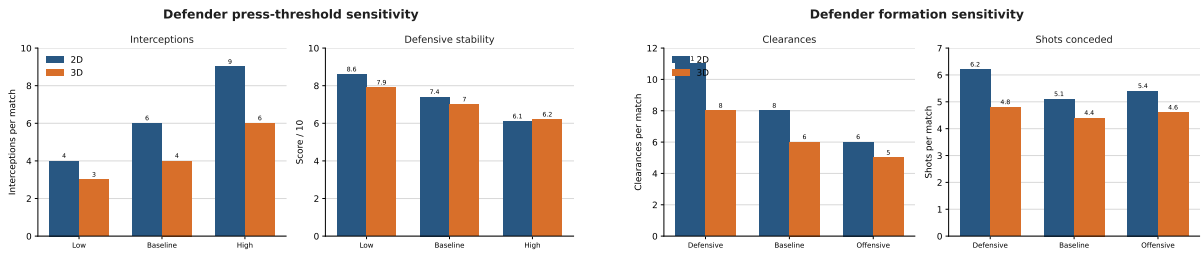
3.5 Defender parameter analysis

The defender-focused results show a trade-off between active ball interruption and defensive stability. Increasing press threshold raised interceptions from 4 to 9 per match in 2D and from 3 to 6 in 3D. However, the defensive stability score fell from 8.6 to 6.1 in 2D and from 7.9 to 6.2 in 3D. The interpretation is that stronger pressure helped the defender contest the ball but also made the defensive shape less stable.

Formation affected defensive workload in a different way. The defensive formation produced the most clearances, with 11 per match in 2D and 8 in 3D, but it did not minimise shots conceded. In both environments, the baseline formation conceded the fewest shots in the benchmark summaries: 5.1 in 2D and 4.4 in 3D. Formation should be interpreted as a spacing and exposure parameter, not as a "more defensive is always better" switch. Table 12 provides the defender values, and figure 5 separates the press-threshold trend in figure 5a from the formation trend in figure 5b.

Table 12: Defender parameter results computed from repeated benchmark runs.

Parameter	Metric	Env.	Low/def.	Baseline	High/off.
Press threshold	Interceptions/match	2D	4	6	9
Press threshold	Interceptions/match	3D	3	4	6
Press threshold	Stability score (/10)	2D	8.6	7.4	6.1
Press threshold	Stability score (/10)	3D	7.9	7.0	6.2
Formation	Clearances/match	2D	11	8	6
Formation	Clearances/match	3D	8	6	5
Formation	Shots conceded/match	2D	6.2	5.1	5.4
Formation	Shots conceded/match	3D	4.8	4.4	4.6



(a) Press-threshold sensitivity. **(b)** Formation sensitivity.

Figure 5: Defender-focused parameter analysis.

3.6 Goalkeeper parameter analysis

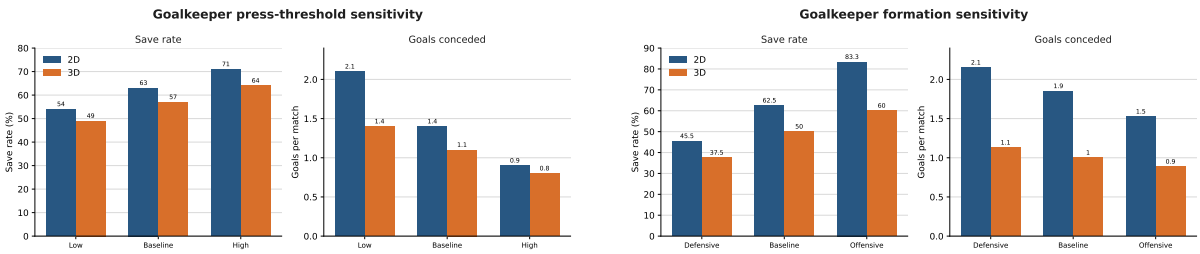
Goalkeeper outcomes depended on the goalkeeper behaviour, upstream team defence, and the number and quality of shots allowed. The goalkeeper results are therefore goalkeeper/team-defence indicators, not a fully isolated goalkeeper controller experiment.

The press-threshold results show that higher pressure was associated with better goalkeeper-facing outcomes. Save rate increased from 54% to 71% in 2D and from 49% to 64% in 3D, while goals conceded decreased from 2.1 to 0.9 in 2D and from 1.4 to 0.8 in 3D. This suggests that pressure changed the quality or volume of shots reaching the goalkeeper, not necessarily that the goalkeeper alone improved.

Formation produced a structural effect. The offensive formation produced the best goalkeeper-facing values, with the highest save rates and lowest goals conceded in both environments. This does not mean the goalkeeper preferred attacking shape in isolation; it indicates that team shape changed the shot exposure conditions faced by the goalkeeper. Table 13 reports these values, and figure 6 compares the pressure trend in figure 6a with the formation trend in figure 6b.

Table 13: Goalkeeper and team-defence results computed from repeated benchmark runs.

Parameter	Metric	Env.	Low/def.	Baseline	High/off.
Press threshold	Save rate (%)	2D	54	63	71
Press threshold	Save rate (%)	3D	49	57	64
Press threshold	Goals conceded/match	2D	2.10	1.40	0.90
Press threshold	Goals conceded/match	3D	1.40	1.10	0.80
Formation	Save rate (%)	2D	45.5	62.5	83.3
Formation	Save rate (%)	3D	37.5	50.0	60.0
Formation	Goals conceded/match	2D	2.15	1.85	1.52
Formation	Goals conceded/match	3D	1.13	1.00	0.89



(a) Press-threshold sensitivity. **(b)** Formation sensitivity.

Figure 6: Goalkeeper and team-defence parameter analysis.

3.7 Cross-role parametric synthesis

Taken together, the parameter-level experiments explained performance better than strategy bundles alone. The striker results were most directly linked to shoot range: higher shoot range increased scoring and reduced time to score in both environments. Defender results showed a trade-off: higher pressure increased interceptions but reduced stability. Goalkeeper-facing results were shaped by upstream pressure and formation, so they should be interpreted as team-defence outcomes rather than goalkeeper-only controller outcomes. Table 14 connects these role-level findings to the final technical interpretation.

Table 14: Cross-role synthesis of the main parameter-level findings.

Role	Dominant parameter(s)	Main interpretation
Striker	**Shooting range; pressing threshold**	Shooting range had the largest effect on attacking output; pressing improved possession and recoveries but with weaker 3D separation.
Defender	**Pressing threshold; formation**	Pressing increased interceptions but reduced stability; formation changed clearance workload and shot exposure.
Goalkeeper	**Pressing threshold; formation / team defence**	Goalkeeper-facing outcomes improved with pressure and varied with formation, indicating strong dependence on upstream defensive structure.

3.8 Interim technical interpretation

The results indicate that broad strategy benchmarking can identify promising configurations, while parameter-level analysis gives a better account of how those configurations differ. Strategy benchmarking identified AGGRO as the best tested bundle. The parameter-level results point to likely mechanisms: more permissive attacking decisions improved striker output, pressure affected both recovery and defensive exposure, and formation changed the shot conditions faced by the goalkeeper.

This interpretation remains bounded by the evidence. The report presents summaries rather than every per-run log row, and the parametric design was local OFAT rather than a full factorial search. OFAT supports local interpretation but does not resolve parameter interactions [10, 11]. Within those limits, the project produced a benchmark workflow that moves from strategy-level comparison to parameter-level behavioural analysis.

4 Team performance analysis

4.1 Team structure and work allocation

The project was divided into workstreams covering simulator onboarding, behaviour development, benchmarking, analysis, presentation, and reporting. This structure was needed because the 2D and 3D stacks had different build paths, runtime assumptions, benchmark scripts, and failure modes. Dedicated 2D and 3D sub-teams handled environment-specific implementation, while the team lead coordinated planning, presentation preparation, report integration, and cross-environment consistency.

The division of labour kept both simulation tracks active, but it also created an integration risk. Results from the 2D and 3D workstreams had to be made comparable, and the report needed a single methodology rather than two disconnected simulator descriptions. This became a coordination task after the design review, when the project shifted from strategy-only benchmarking to parameter-level evidence. Table 15 relates each workstream to the management observation it produced.

The team also used standard software-engineering practices to support coordination and maintainability. Git-based version control helped the 2D and 3D workstreams progress in parallel while preserving a shared integration history. Repository documentation, benchmark command notes, and generated Doxygen API documentation reduced onboarding friction and made the software artefacts easier to audit. Automated Doxygen deployment through GitHub Actions further supported quality control by making API documentation generation repeatable rather than manually maintained.

Table 15: Main project workstreams and management observations.

Workstream	Main focus	Management observation
Simulator setup / onboarding	Establish usable 2D and 3D runtime paths.	More time-consuming than estimated because inherited stacks required compatibility work and environment-specific troubleshooting.
Behaviour development	Implement strategy mappings, role logic, and parameter controls.	Progress was steady once the simulators were usable, but behaviour changes still required repeated validation.
Strategy benchmarking	Compare whole behaviour bundles against BASIC.	Closest workstream to the original estimate because the scope was clear once scripts were available.
Parametric benchmarking	Run OFAT tests for shoot range, press threshold, and formation.	Required more discipline than expected because each run had to preserve a fixed baseline and vary only one parameter.
Analysis and interpretation	Convert outputs into plots, tables, and role-level explanation.	Largest underestimation; converting benchmark outputs into defensible conclusions took more effort than raw match execution.
Documentation and version control	Maintain shared repository history, setup notes, benchmark command documentation, and generated API documentation.	Improved coordination and reproducibility, especially when work was split across simulator-specific sub-teams.
Presentation / reporting	Produce design-review, final-presentation, and final-report material.	Increased after the methodology changed and the report needed a stronger evidence narrative.

4.2 Planning, review, and iteration

The project followed an iterative engineering workflow informed by weekly supervisory meetings, milestone reviews, and formal presentation checkpoints. The planned work-breakdown schedule is shown in figure 7. It is kept at work-package level rather than detailed subtask level, using a V-model/waterfall-style progression from requirements and platform setup through implementation, verification, analysis, and reporting. The dates are anchored to the module schedule and TAR evidence, from the October 2025 kick-off through the April 2026 final submission window.

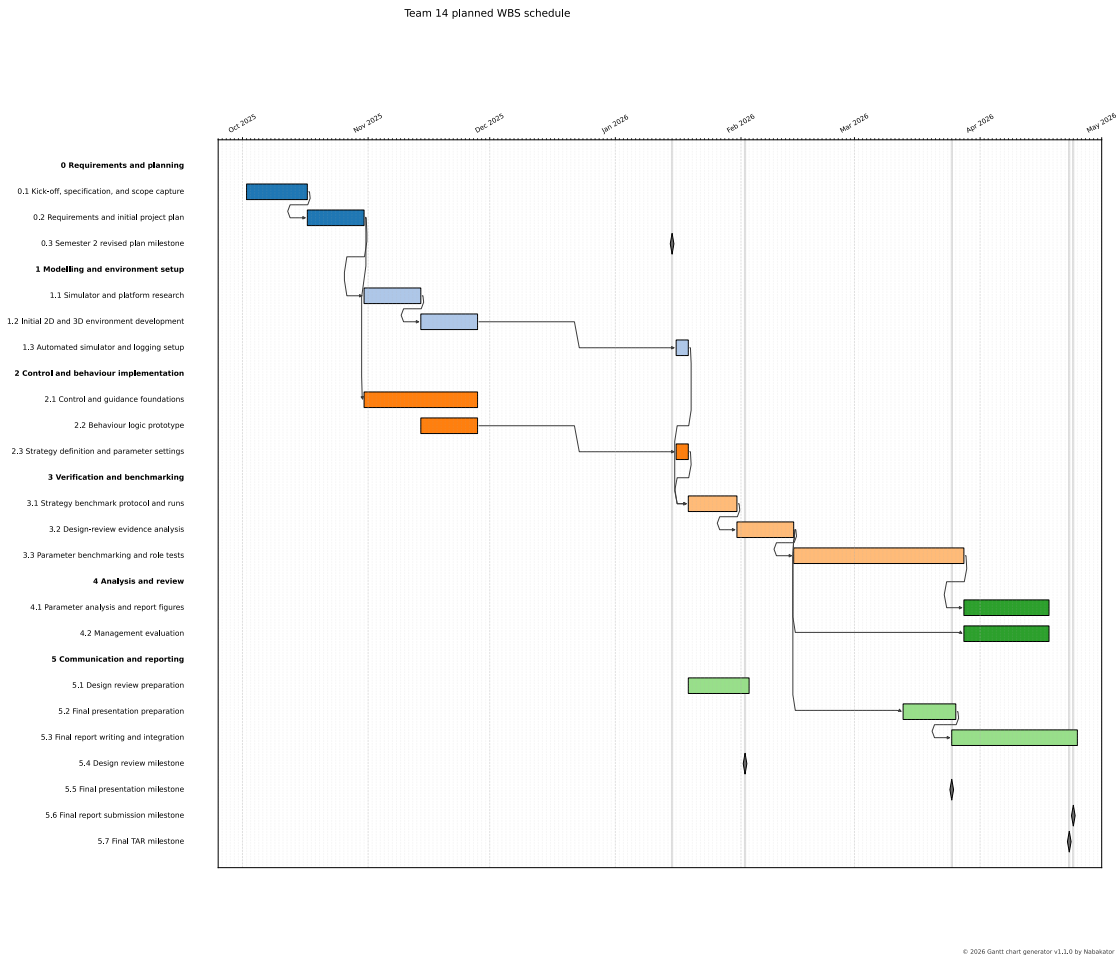


Figure 7: Planned high-level WBS Gantt chart for the Stage 1 project schedule, grouped by major work package.

The design review stage changed the project plan. Feedback from Dr Euan McGookin and Ziang Cheng helped the team recognise an oversight in the original evaluation plan: broad strategy-level benchmarking was not explanatory enough without isolated parameter-level analysis. The project plan was revised to add OFAT parameter-isolation experiments and role-informed interpretation.

This decision improved the technical argument, but it increased the workload. Strategy benchmarking mainly required running complete matches and comparing outcomes. Parametric benchmarking required a fixed baseline, one parameter varied at a time, consistent levels, mode selection, and interpretation by role. The project became more analytically demanding than the initial plan assumed.

4.3 Estimated versus actual time and cost

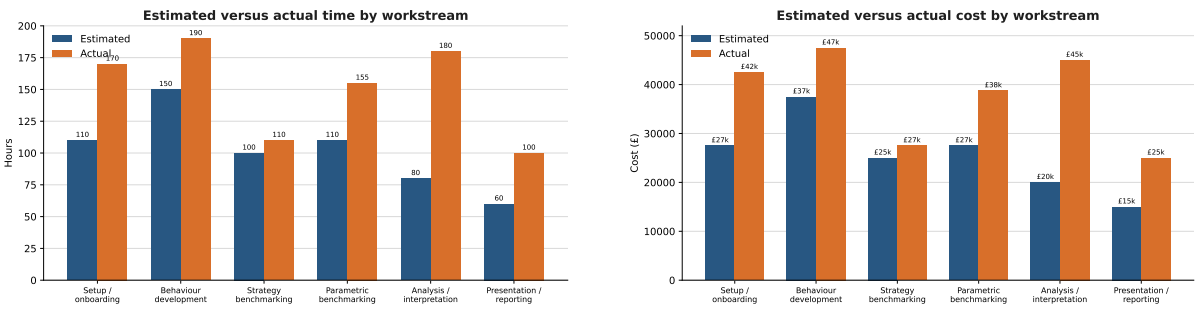
Time and cost estimates were developed at the beginning of the project and updated through the final project records. The cost figures in this report are labour-equivalent estimates calculated at a fixed nominal rate of £250 per hour; they should therefore be interpreted as project-management accounting rather than external cash expenditure.

The initial estimate was 610 hours, corresponding to £152,500. The final recorded effort was 905 hours, corresponding to £226,250. This is an increase of 295 hours, or approximately 48%, and an equivalent labour-cost increase of £73,750.

The largest overrun was analysis and interpretation, which increased from 80 to 180 hours. Collecting match outputs was only part of the work; converting them into defensible conclusions required data checking, plotting, interpretation, and report alignment. Simulator setup and onboarding was the second-largest overrun, increasing by 60 hours, because inherited software stacks required more compatibility and workflow repair than expected. The workstream-level figures are shown in table 16 and plotted in figure 8: figure 8a shows the time overrun by workstream and figure 8b converts the same pattern into labour-equivalent cost. Figure 9 then reduces the same evidence to the overall estimate-versus-actual project overrun.

Table 16: Estimated and actual effort by workstream from the project records.

Workstream	Estimated h	Actual h	Difference h	Actual cost
Simulator setup / onboarding	110	170	+60	£42,500
Behaviour development	150	190	+40	£47,500
Strategy benchmarking	100	110	+10	£27,500
Parametric benchmarking	110	155	+45	£38,750
Analysis and interpretation	80	180	+100	£45,000
Presentation / reporting	60	100	+40	£25,000
Total	610	905	+295	£226,250



(a) Estimated versus actual time by workstream. **(b)** Estimated versus actual cost by workstream.

Figure 8: Estimated and actual project effort and labour-equivalent cost by workstream.

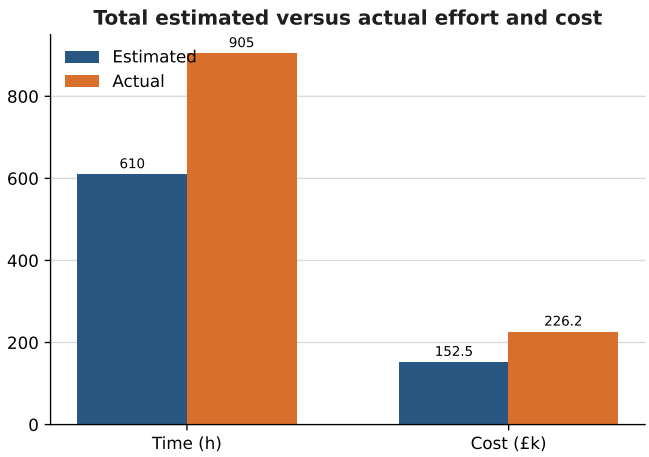


Figure 9: Total estimated versus actual effort and labour-equivalent cost.

4.4 Management evaluation against assessment criteria

The project specification highlights stewardship, teamwork, systems thinking, leadership, quality/risk focus, and change management as central project-management concerns, while the assessment matrix places particular weight on planning, initiative, teamwork, and technical quality [1, 21]. Using those criteria, the team's performance can be evaluated as shown in table 17.

Table 17: Evaluation of project performance against the management criteria implied by the brief and assessment matrix.

Dimension	Evidence from the project	Assessment	Main lesson
Planning / stewardship	Work was divided into simulator onboarding, behaviour development, benchmarking, analysis, and reporting, with TAR-based time recording and a final estimate-versus-actual comparison.	The team created a workable management structure, but effort estimation was materially optimistic, especially for simulator setup and post-processing.	Early plans should treat integration and analysis as primary workstreams rather than end-of-project overhead.
Initiative / change management	After the design review, the team revised the technical plan from broad strategy screening to parameter-level isolation and role-based interpretation.	This was the key management decision in the project because it improved the technical argument rather than preserving the original scope.	Mid-project review gates should be used to improve evidential quality, even when they increase workload.
Teamwork / systems integration	Dedicated 2D and 3D workstreams kept both environments active, while the team lead coordinated cross-environment consistency and report integration.	The team structure was functional and productive, but it created a real integration burden because the environments differed in build paths, runtime assumptions, and output characteristics.	Parallel workstreams need explicit integration checkpoints, not just separate progress.
Technical quality / risk focus	Stage 1 delivered usable simulators, autonomous behaviour logic, benchmark automation, and role-level analysis, but Stage 2 hardware transfer was not reached.	The technical outcome satisfies Stage 1, but external validity remains limited because the simulation-to-hardware step was not completed.	Scope boundaries should be stated explicitly; partial completion is stronger when it is honestly bounded and well evidenced.

4.5 Evaluation of team performance

From a project-management perspective, the team's performance was stronger in adaptation and technical judgement than in initial estimation. The group maintained a functioning division of labour across simulator understanding, behaviour development, benchmarking, and reporting, and that structure was sufficient to complete Stage 1. However, the initial plan understated the effort required to make inherited environments usable and to transform raw benchmark outputs into a defensible technical narrative.

The decisive positive feature of the project was the willingness to revise the scope after the design review. Instead of treating the pitch presentation as a box-ticking milestone, the team used it as a review gate and shifted towards parameter-level evidence. That change increased the workload, but it also made the final report more aligned with the engineering purpose of the assignment. The main weakness remained forecasting: environment setup, cross-environment reconciliation, and analysis all consumed more resource than the original plan assumed. The team also did not reach Stage 2 hardware implementation, which should be understood as an explicit scope boundary rather than concealed behind the simulation results.

4.6 Lessons learned

The project showed that performance evaluation is an engineering problem in its own right. A functioning simulation is not enough; the project also needs a benchmarking framework that produces interpretable conclusions. The work improved when it moved away from a demonstration mindset and towards an evidence mindset.

The main planning lesson is that analysis and integration should be estimated as workstreams, not as small final tasks after implementation. A second lesson is that stage gates need explicit success criteria: for a project of this kind, "simulation complete" should mean runnable environments, autonomous behaviours, repeated benchmark evidence, and a traceable analysis path, not simply a visible match. For future work, the team would allocate more contingency to environment setup, preserve a clearer raw-data archive alongside summary tables, define the parameter-level experiment matrix earlier, and articulate the Stage 1 to Stage 2 transfer requirements earlier. These changes would reduce late-stage integration pressure and make the final evidence package easier to audit.

5 Conclusions and further work

5.1 Conclusions

This report has presented the development and evaluation of a simulation-based benchmarking framework for cyberphysical RoboCup soccer teams. Stage 1 was completed in the sense that both 2D and 3D simulation environments were used to generate comparative evidence on team behaviour and role-related process indicators. Strategy-level comparison identified AGGRO as the best behaviour bundle among the tested configurations, while parameter-level analysis made the performance differences easier to interpret. The striker results link attacking output

to shooting-range settings, defender indicators were shaped by pressure-related parameters and formation effects, and goalkeeper outcomes were influenced by the defensive structure that determined shot exposure. Although Stage 2 hardware implementation was not completed, the project delivered a simulation and analysis foundation for future transfer, with the usual caution that simulation results require validation before physical deployment [9, 18].

5.2 Overall assessment of project outcomes

Overall, the project met the main technical objective: a behavioural benchmarking pipeline rather than a visual simulation alone. The main technical contribution is the shift from broad strategy comparison to parameter-level analysis. That makes the framework more informative than a tournament-style comparison of strategies.

5.3 Further work

Future work should be interpreted against two constraints already visible in the present study: policies that perform well in simulation still face a reality gap when transferred onto physical robots [18, 22]; and a NAO6 deployment would need to respect the sensing, actuation, computing, and physical constraints of the target platform rather than assume that simulator behaviours transfer unchanged [23].

Several directions for further work follow from the present study:

1. transfer the selected behavioural logic to physical NAO6 hardware platforms, while explicitly addressing the simulation-to-reality gap identified in humanoid RoboCup work [18];
2. introduce onboard perception using camera-based ball and agent detection;
3. refine the mapping between idealised simulation actions and NAO-compliant motion and balance control;
4. extend the parameter search space beyond the three isolated behavioural factors used here;
5. increase statistical strength through larger benchmark sets and richer uncertainty quantification, following the repeated-evaluation concerns raised for RoboCup simulation leagues [8, 20]; and
6. investigate whether a more unified cross-environment evaluation framework can better reconcile the separation observed in 2D with the draw-dominated behaviour seen in 3D.

References

[1] Euan McGookin. Robotics Team Design Project M (ENG5325): Cyberphysical RoboCup Soccer Teams specification document 2025–2026, 2025. Module specification document, James Watt School of Engineering, University of Glasgow.

[2] Hiroaki Kitano, Minoru Asada, Yasuo Kuniyoshi, Itsuki Noda, Eiichi Osawa, and Hitoshi Matsubara. RoboCup: A challenge problem for AI. *AI Magazine*, 18(1):73–85, 1997.

[3] Hiroaki Kitano, Milind Tambe, Peter Stone, Manuela Veloso, Silvia Coradeschi, Eiichi Osawa, Hitoshi Matsubara, Itsuki Noda, and Minoru Asada. The RoboCup synthetic agent challenge 97. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*, pages 24–29, San Francisco, CA, 1997. Morgan Kaufmann.

[4] Daniele Nardi, Itsuki Noda, Fernando Ribeiro, Peter Stone, Oskar von Stryk, and Manuela Veloso. RoboCup soccer leagues. *AI Magazine*, 35(3):77–85, 2014. doi: 10.1609/aimag.v35i3.2549.

[5] RoboCup Humanoid League Technical Committee. Call for participation – RoboCup 2025 humanoid soccer competition, 2024. URL https://humanoid.robocup.org/robocup-2025/call-for-participation/. Accessed 24 April 2026.

[6] RoboCup Humanoid League. RoboCup humanoid league laws of the game 2025, 2025. URL https://humanoid.robocup.org/wp-content/uploads/RC-HL-2025-Rules.pdf. Accessed 24 April 2026.

[7] RoboCup Standard Platform League. RoboCup standard platform league: New-home, 2025. URL https://spl.robocup.org/new-home/. Accessed 24 April 2026.

[8] David M. Budden, Peter Wang, Oliver Obst, and Mikhail Prokopenko. RoboCup simulation leagues: Enabling replicable and robust investigation of complex robotic systems. *IEEE Robotics & Automation Magazine*, 22(3):140–146, 2015. doi: 10.1109/MRA.2015.2446911.

[9] HeeSun Choi, Cindy Crump, Christian Duriez, Asher Elmquist, Gregory Hager, David Han, Frank Hearl, Jessica Hodgins, Abhinandan Jain, Frederick Leve, Chen Li, Franziska Meier, Dan Negrut, Ludovic Righetti, Alberto Rodriguez, Jie Tan, and Jeff Trinkle. On the use of simulation in robotics: Opportunities, challenges, and suggestions for moving forward. *Proceedings of the National Academy of Sciences of the United States of America*, 118(1): e1907856118, 2021. doi: 10.1073/pnas.1907856118.

[10] Andrea Saltelli, Marco Ratto, Stefano Tarantola, and Francesca Campolongo. Sensitivity analysis practices: Strategies for model-based inference. *Reliability Engineering & System Safety*, 91(10–11):1109–1125, 2006. doi: 10.1016/j.ress.2005.11.014.

[11] Guus ten Broeke, George van Voorn, and Arend Ligtenberg. Which sensitivity analysis method should i use for my agent-based model? *Journal of Artificial Societies and Social Simulation*, 19(1):5, 2016. doi: 10.18564/jasss.2857.

[12] Euan McGookin. Robotics Team Design Project M (ENG5325): Report and Presentation Guidelines 2026, 2026. Module guidance document, James Watt School of Engineering, University of Glasgow.

[13] Itsuki Noda and Peter Stone. The RoboCup soccer server and CMUnited clients: Implemented infrastructure for MAS research. *Autonomous Agents and Multi-Agent Systems*, 7 (1–2):101–120, 2003. doi: 10.1023/A:1024128904944.

[14] The RoboCup Soccer Simulator Maintenance Committee. The RoboCup soccer simulator users manual, 2024. URL https://rcsoccersim.readthedocs.io/en/latest/. Last updated 25 March 2024; accessed 24 April 2026.

[15] Hidehisa Akiyama and Tomoharu Nakashima. HELIOS base: An open source package for the RoboCup soccer 2d simulation. In *RoboCup 2013: Robot World Cup XVII*, volume 8371 of *Lecture Notes in Computer Science*, pages 528–535. Springer, 2014. doi: 10.1007/978-3-662-44468-9_46.

[16] Justin Stoecker and Ubbo Visser. Visualizing and debugging complex multi-agent soccer scenes in real time. In *RoboCup 2013: Robot World Cup XVII*, volume 8371 of *Lecture Notes in Computer Science*, pages 640–647. Springer, 2014. doi: 10.1007/978-3-662-44468-9_60.

[17] Shivaram Kalyanakrishnan, Todd Hester, Michael Quinlan, Yinon Bentor, and Peter Stone. Three humanoid soccer platforms: Comparison and synthesis. In *RoboCup 2009: Robot Soccer World Cup XIII*, volume 5949 of *Lecture Notes in Computer Science*, pages 140–152. Springer, 2010. doi: 10.1007/978-3-642-11876-0_14.

[18] Yuan Xu, Heinrich Mellmann, and Hans-Dieter Burkhard. An approach to close the gap between simulation and real robots. In *Simulation, Modeling, and Programming for Autonomous Robots*, volume 6472 of *Lecture Notes in Computer Science*, pages 533–544. Springer, 2010. doi: 10.1007/978-3-642-17319-6_48.

[19] Nuno Lau, Luis Seabra Lopes, Gustavo Corrente, and Nelson Filipe. Robot team coordination using dynamic role and positioning assignment and role based setplays. *Mechatronics*, 21(2):445–454, 2011. doi: 10.1016/j.mechatronics.2010.05.010.

[20] Mikhail Prokopenko, Peter Wang, Sebastian Marian, Aijun Bai, Xiao Li, and Xiaoping Chen. RoboCup 2d soccer simulation league: Evaluation challenges. In *RoboCup 2017: Robot World Cup XXI*, volume 11175 of *Lecture Notes in Artificial Intelligence*, pages 325–337. Springer, 2018.

[21] Euan McGookin. Assessment Matrix for Robotics Team Project M 2026, 2026. Assessment criteria document, James Watt School of Engineering, University of Glasgow.

[22] Fabio Muratore, Fabio Ramos, Greg Turk, Wenhao Yu, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9: 799893, 2022. doi: 10.3389/frobt.2022.799893.

[23] Aldebaran. NAO V6 Datasheet, 2024. URL https://aldebaran.com/library/NAO6/NAO%20V6%20Datasheet_EN_ALD%202024-2025%20%281%29.pdf. Version 10/2024; accessed 24 April 2026.

A Benchmark command sheet

The following commands record the benchmark workflows used in the project. They are included as reproducibility notes rather than as a claim that every full run was repeated immediately before submission. The project repository is available at https://github.com/ENG5325-Robotics-TDP-M-Team-14/Cyberphysical-RoboCup-Soccer-Teams.git.

A.1 2D commands

The 2D commands are run from:

```
environment/2d-environment/starter-stack
```

The 2D strategy benchmark is the fixed 4v4 suite:

```
./run_strategy_benchmark_2d.sh
```

The preferred isolated 2D parametric mode is 2v2, because it includes one field player and one goalkeeper per side. A full shoot-range sweep can be launched with:

```
./run_parametric_benchmark_2d.sh \
  --parameter shoot_range \
  --mode 2v2 \
  --levels low,baseline,high \
  --repeats 5
```

The no-goalie 1v1 microbenchmark remains available for reduced field-player checks:

```
./run_parametric_benchmark_2d.sh \
  --parameter press_threshold \
  --mode 1v1 \
  --levels low,baseline,high \
  --repeats 5
```

The 4v4 parametric mode is used when a full small-team context is required:

```
./run_parametric_benchmark_2d.sh \
  --parameter formation \
  --mode 4v4 \
  --levels baseline,def,off \
  --repeats 5
```

A short smoke-test command used to validate the 2D pipeline is:

```
RCSSSERVER_PORT_BASE=7900 MATCH_TIMEOUT_SECONDS=20 \
START_DELAY=1 SIDE_DELAY=1 HALF_TIME_CYCLES=30 \
```

```
./run_parametric_benchmark_2d.sh \
  --parameter shoot_range \
  --mode 2v2 \
  --levels low \
  --repeats 1
```

A.2 3D commands

The 3D commands are run from:

```
environment/3d-environment/scripts
```

The 3D strategy benchmark is the fixed 4v4 suite:

```
BENCH_NO_INHIBIT=1 ./run_strategy_benchmark_3d.sh
```

The 3D parametric script supports 1v1, 2v2, and 4v4 modes. A 1v1 shoot-range sweep can be launched with:

```
BENCH_NO_INHIBIT=1 ./run_parametric_benchmark_3d.sh \
  --parameter shoot_range \
  --mode 1v1 \
  --levels low,baseline,high \
  --repeats 5 \
  --half-time-timeout-sec 420
```

The 2v2 mode includes one field player and one goalkeeper per side:

```
BENCH_NO_INHIBIT=1 ./run_parametric_benchmark_3d.sh \
  --parameter shoot_range \
  --mode 2v2 \
  --levels low,baseline,high \
  --repeats 5 \
  --half-time-timeout-sec 420
```

The 4v4 mode is used for fuller team-level parametric checks:

```
BENCH_NO_INHIBIT=1 ./run_parametric_benchmark_3d.sh \
  --parameter formation \
  --mode 4v4 \
  --levels baseline,def,off \
  --repeats 5 \
  --half-time-timeout-sec 420
```

A short 3D validation command used during pipeline testing is:

```
BENCH_NO_INHIBIT=1 MATCH_WALL_TIMEOUT_SEC=900 \
PROGRESS_INTERVAL_SEC=30 ./run_parametric_benchmark_3d.sh \
  --parameter shoot_range \
  --mode 2v2 \
  --levels low \
  --repeats 1 \
  --half-time-timeout-sec 420
```

B Benchmark output structure

The parametric scripts write deterministic run directories under simulator-specific benchmark output roots. Each run receives an incrementing identifier such as `run_0001`, `run_0002`, and so on. The output roots for the strategy and parametric workflows are listed in table 18.

Table 18: Benchmark output locations.

Workflow	Output location
2D strategy	`environment/2d-environment/starter-stack/strategy_benchmark_results_2d.csv` and `strategy_benchmark_logs_2d/`.
2D parametric	`environment/2d-environment/starter-stack/benchmark_outputs/2d/parametric/mode_<mode>/baseline_BASIC/parameter_<parameter>/run_<index>/`.
3D strategy	`environment/3d-environment/strategy_benchmark_logs_3d/strategy_benchmark_results_3d.csv`, unless an explicit `--out-csv` or `--log-dir` is provided.
3D parametric	`environment/3d-environment/benchmark_outputs/3d/parametric/mode_<mode>/baseline_BASIC/parameter_<parameter>/run_<index>/`.

For parametric runs, the generated files are summarised in table 19.

Table 19: Common files generated by parametric benchmark runs.

File or directory	Purpose
`results.csv`	Combined match rows for the run, including simulator, mode, parameter, level, teams, goals, status, and error information.
`levels.csv`	Parameter levels evaluated during the run and their mapped internal strategy/configuration values.
`run_metadata.json`	Run identifier, output root, mode, baseline controller, parameter name, port settings where relevant, and timestamp metadata.
`metrics_catalog.csv`	Catalogue of available metrics and expected metric provenance.
`behavioural_metrics_scaffold.csv`	Reserved structure for richer behavioural metrics where parser-level extraction is not yet complete.
`match_logs/`, `server_logs/`, or `levels/`	Simulator-specific logs and intermediate strategy outputs used for audit and debugging.

C Benchmark summary tables

The following tables reproduce the summarised values used for the main report figures. They make the numerical basis visible inside the report. Tables 20 to 23 correspond respectively to the strategy, striker, defender, and goalkeeper/team-defence summaries discussed in the main results section.

Table 20: Strategy benchmark summary against BASIC.

Strategy	2D goal difference	3D goal difference	2D win rate (%)	3D win rate (%)
NOISE	0.0	0.0	50	33
DEFLOCK	0.4	0.1	56	36
HIPRESS	0.8	0.2	64	40
DIRECT	0.6	0.1	60	37
AGGRO	1.4	0.3	76	46

Table 20 is the appendix version of the strategy data used in table 9 and figure 3.

Table 21: Striker parameter summaries.

Parameter	Level	2D metric A	3D metric A	2D metric B	3D metric B
Shoot range	Low	0.8	0.3	68	95
Shoot range	Baseline	1.2	0.5	58	82
Shoot range	High	1.7	0.8	46	70
Press threshold	Low	42	47	7	5
Press threshold	Baseline	51	50	10	7
Press threshold	High	60	54	14	9

In table 21, metric A is goals per match for shoot range and possession percentage for press threshold. Metric B is time to score in seconds for shoot range and ball recoveries per match for press threshold.

Table 22: Defender parameter summaries.

Parameter	Level	2D metric A	3D metric A	2D metric B	3D metric B
Press threshold	Low	4	3	8.6	7.9
Press threshold	Baseline	6	4	7.4	7.0
Press threshold	High	9	6	6.1	6.2
Formation	Defensive	11	8	6.2	4.8
Formation	Baseline	8	6	5.1	4.4
Formation	Offensive	6	5	5.4	4.6

In table 22, metric A is interceptions per match for press threshold and clearances per match for formation. Metric B is defensive stability score out of 10 for press threshold and shots conceded per match for formation.

Table 23: Goalkeeper and team-defence parameter summaries.

Parameter	Level	2D save rate (%)	3D save rate (%)	2D goals conceded	3D goals conceded
Press threshold	Low	54.0	49.0	2.10	1.40
Press threshold	Baseline	63.0	57.0	1.40	1.10
Press threshold	High	71.0	64.0	0.90	0.80
Formation	Defensive	45.5	37.5	2.15	1.13
Formation	Baseline	62.5	50.0	1.85	1.00
Formation	Offensive	83.3	60.0	1.52	0.89

Table 23 is the appendix version of the goalkeeper/team-defence values used in table 13 and figure 6.

D Workflow notes and limitations

The benchmarking workflow uses strategy names as a compatibility layer between scripts and controller configurations. For strategy benchmarking, each named strategy represents a bundle of behaviour settings. For parametric benchmarking, the scripts vary exactly one target parameter at a time while keeping the baseline controller fixed as BASIC. This is the practical implementation of the OFAT methodology described in section 2.

The reduced-player modes have different meanings. The 1v1 mode is a no-goalie duel for lightweight field-player checks. The 2v2 mode is the preferred isolated project-style mode because each side includes one field player and one goalkeeper, making it closer to the 4v4 setup. The 4v4 mode preserves the fuller team context and is most appropriate for formation-related interpretation.

The 2D workflow depends on the CMake-based 2D build path and the compatibility bridge that links the modern `sample_player`, `sample_coach`, and `sample_trainer` binaries into the legacy starter-stack script location. The legacy autotools build path under the old starter-stack source is not treated as the supported build route.

The 3D workflow may use RoboViz-visible execution during benchmarking. The project does not require a fully headless 3D workflow; the requirement is that one script can launch the benchmark and collect the relevant output data. Timeout settings such as `-half-time-timeout-sec` and `MATCH_WALL_TIMEOUT_SEC` are practical safeguards because the 3D simulator may progress more slowly in wall-clock time than in simulator time.